Vector Databases

Querying Similarity

Week 13, Part 1

PPOL 5206 — Massive Data Fundamentals • Spring 2026

Georgetown University — McCourt School of Public Policy

Today's Roadmap

01 **The Database Landscape**
Where do vector databases fit among the paradigms we've studied?

02 **From Words to Numbers**
Embeddings, vector spaces, and why similarity matters

03 **How Vector Search Works**
ANN algorithms, HNSW, and the accuracy-speed tradeoff

04 **The Vector Database Ecosystem**
Pinecone, Milvus, pgvector, and the convergence trend

05 **RAG and LLM Infrastructure**
Why every AI application needs a vector layer

06 **Implications for Policy Data**
Public sector use cases and decision frameworks

The Database Paradigm Landscape

Each paradigm optimizes for a different binding constraint

Relational

Structured queries, ACID

PostgreSQL, MySQL

Wide Column

Write-heavy time series

Cassandra, HBase

Document

Flexible schema, reads

MongoDB, DynamoDB

Graph

Relationships, traversals

Neo4j, Neptune

Key-Value

Speed, simple lookups

Redis, Elasticache

Vector

Similarity, semantics

Pinecone, Milvus

01

From Words to Numbers

The embedding revolution that made similarity searchable

What Is an Embedding?

A numerical representation of meaning in high-dimensional space

The Core Idea

Text, images, and other unstructured data are converted into dense vectors — arrays of floating-point numbers (typically 384–1,536 dimensions).

Similar concepts cluster together in this vector space. "Cat" and "kitten" are nearby; "cat" and "invoice" are far apart.

This transforms a semantic question ("what does this mean?") into a geometric one ("what's nearby?").

Example: Text Embedding

```
"How do I reset my password?"
→ [0.023, -0.147, 0.892, …, 0.041]

"Steps for password change"
→ [0.019, -0.153, 0.887, …, 0.038]
```

`cosine_similarity ≈ 0.97`

These phrases share no exact words, yet they're semantically near-identical.

How Embeddings Are Learned

Word2Vec uses a shallow neural network to discover meaning from context

The Network Architecture

Three layers — minimum for a hidden layer:

Input	One-hot vector : vocab size (~50K)
Hidden	100–300 neurons (linear)
Output	Softmax over full vocabulary

The hidden layer is the embedding.

The Key Insight

Network is trained to predict context: given ***"cat"***, predict ***"the"***, ***"sat"***, ***"mat"***.

But we don't care about the predictions.

We care what the hidden layer had to learn.

To compress 50,000 one-hot dims into 300 dense ones and still predict context, those 300 dimensions must encode meaning.

After training, discard the output layer. The input→hidden weights are a lookup table of word vectors.

Vocabulary Size in Perspective

How big are the vocabularies we're working with?

Human

Newspaper

15–20K

covers ~98%
of typical text

Human

High School Grad

25–30K

word families
(groups inflections)

Human

Educated Adult

40–60K

word families;
individual forms higher

NLP

TF-IDF Pipeline

50–100K

typical max_features
(one dim per word)

NLP

GPT Tokenizer

~50K

subword BPE tokens
('un'+'forget'+'table')

NLP

Google News W2V

3M

trained on ~100B
word tokens

When TF-IDF builds a 50K-term vocabulary, that's roughly one adult's active vocabulary — and each word is its own dimension. This is why dense 300-dim or 768-dim embeddings are such a dramatic compression.

Do We Clean the Text First?

Word2Vec preprocessing is less aggressive than you'd expect

Typically Done

Lightweight normalization, not semantic changes

- **Tokenization**
  Split on whitespace & punctuation
- **Lowercasing**
  Usually — though Google News kept case
- **Frequency thresholding**
  Drop words appearing < ~5 times
- **Subsampling**
  Randomly discard high-frequency words

Typically Avoided

These destroy info the model can learn itself

- **Stemming**
  "running" → "run" loses useful distinctions
- **Lemmatization**
  Model already learns inflected forms cluster
- **Stop-word removal**
  Skip-gram naturally downweights them
- **Heavy normalization**
  Case, punctuation, numbers carry signal

Contrast: *TF-IDF pipelines often DO stem and lemmatize — because they can't learn relationships on their own.*

A Brief History of Embeddings

2013 — **Word2Vec**

Mikolov et al. at Google show words can be vectors; "king − man + woman ≈ queen"

2017 — **Transformers**

"Attention Is All You Need" — contextual embeddings replace static word vectors

2019 — **Milvus & Pinecone**

First purpose-built vector databases appear; ANN search goes production-grade

2020 — **FAISS at Scale**

Meta's FAISS library enables billion-scale similarity search

2022 — **ChatGPT + RAG**

LLM explosion drives massive demand for vector retrieval infrastructure

2024–26 — **Convergence**

pgvector, Oracle 23ai, MongoDB Atlas add native vector support; vectors become a data type

Relational vs. Vector: Different Questions

Relational Database

Stores: Structured rows & columns
Query: Exact match & conditions
Example: "Find all users where city = 'DC' AND age > 30"
Strength: ACID transactions, joins, aggregations
Weakness: Cannot query meaning, similarity, or intent

Vector Database

Stores: High-dimensional embeddings
Query: Similarity / nearest neighbor
Example: "Find the 5 documents most similar to this question"
Strength: Semantic search, handles unstructured data
Weakness: No joins, no ACID, approximate results

02

How Vector Search Works

From brute force to billion-scale approximate nearest neighbors

A Landscape of Similarity

Three different questions, three different toolkits

Syntactic

How many keystrokes apart?

Tools

Levenshtein, Jaro-Winkler
Soundex, pg_trgm

Use cases

Spell-check, fuzzy match,
record linkage

`"colour" ≈ "color"`

Statistical

How many shared words?

Tools

TF-IDF, BM25, Jaccard

Use cases

Keyword search,
traditional retrieval

`"cat sat mat" ≠ "kitten rested rug"`

Semantic

How close in meaning?

Tools

Cosine, L2 over
dense embeddings

Use cases

RAG, semantic search,
recommendations

`"reset password" ≈ "change password"`

All three live in your toolkit. *Choose by the question you're asking — not the data type.*

Edit Distance in Practice

When syntactic similarity is exactly what you want

When to Reach for It

- **Entity resolution**
  "John A. Smith" ≈ "Smith, John"
- **Deduplication**
  Same record entered twice with typos
- **Spell checking**
  Find closest valid word in a dictionary
- **Fuzzy joins**
  Match across messy government datasets

Tools (already in your RDS!)

Levenshtein — `fuzzystrmatch`

Insertions, deletions, substitutions

Jaro-Winkler — `fuzzystrmatch`

Edit distance + prefix bonus (good for names)

Soundex / Metaphone — `fuzzystrmatch`

Phonetic matching: "Smith" ≈ "Smyth"

Trigram similarity — `pg_trgm`

Character n-gram overlap, indexable

→ ***Edit distance is also how GraphRAG resolves entities*** *("Dr. Smith" the oncologist vs. "Dr. Smith" the cardiologist) — see Part 2.*

Distance Metrics: How "Close" Are Two Vectors?

Cosine Similarity

$\cos(\theta) = A \cdot B / (||A|| \times ||B||)$

Text & NLP embeddings — measures angle between vectors, ignoring magnitude. Most common default.

Euclidean (L2)

$d = \sqrt{\Sigma(a_i - b_i)^2}$

Image embeddings, spatial data — measures straight-line distance in vector space.

Dot Product

$d = \Sigma(a_i \times b_i)$

Normalized embeddings — equivalent to cosine when vectors are unit-length. Fastest to compute.

Cosine Similarity, Derived

Why the formula looks the way it does

Start: Geometric Intuition

$$\cos(\theta) = \text{adjacent} / \text{hypotenuse}$$

Familiar from right triangles, but we can't draw one in 300 dimensions. We need an algebraic formula.

Bridge: The Dot Product Identity

$$a \cdot b = \|a\| \, \|b\| \cos(\theta)$$

From linear algebra: dot product = product of magnitudes times cos of the angle. Provable via the law of cosines.

Rearrange to solve for cosine:

$$\cos(\theta) = (a \cdot b) / (\|a\| \, \|b\|)$$

Why this for text? The denominator normalizes away magnitude. A tweet and a news article on the same topic have different vector lengths but similar ***directions*** — and direction is what captures "aboutness."

Approximate Nearest Neighbors (ANN)

The speed-accuracy tradeoff at the heart of vector search

The Problem

Exact nearest neighbor search (brute force) compares your query against every vector in the database. At 1 billion vectors × 1,536 dimensions, this is computationally impossible in real time.

The Solution: ANN

ANN algorithms build index structures that dramatically prune the search space. You sacrifice a tiny amount of accuracy (e.g. 99% recall vs. 100%) for 1000x speedups. Results in 10–50ms.

HNSW

Hierarchical Navigable Small World graphs. Most popular. Low latency, high recall. Used by Milvus, Qdrant, Weaviate, pgvector.

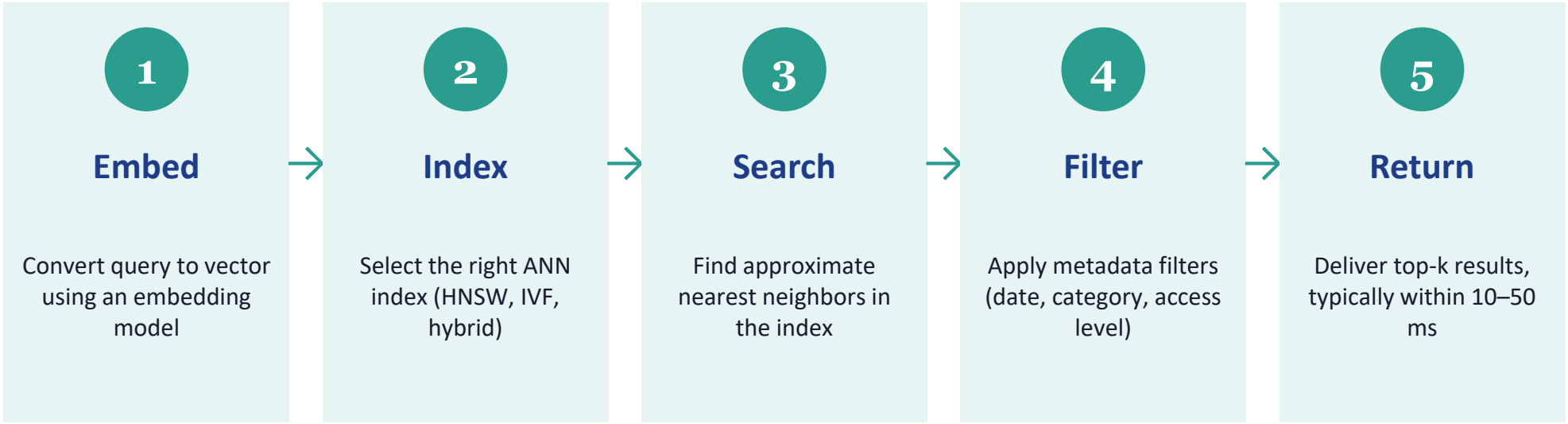
IVF

Inverted File Index. Clusters vectors into groups, searches only relevant clusters. Good for very large datasets.

PQ

Product Quantization. Compresses vectors to reduce memory. Combines with HNSW/IVF for billion-scale search on disk.

The Vector Search Pipeline



This entire cycle happens on every query — and it must complete in milliseconds for production use.

03

The Vector Database Ecosystem

Purpose-built vs. bolt-on — and why it matters less than you think

Purpose-Built Vector Databases

Pinecone *Fully managed*	Simplest setup, enterprise SLAs, serverless. The "it just works" option.	`Proprietary`
Milvus *Open-source*	Handles billions of vectors. Distributed, cloud-native. LF AI & Data Foundation project.	`Apache 2.0`
Qdrant *Open-source*	Rust-based, high performance. Strong payload filtering for hybrid queries.	`Apache 2.0`
Weaviate *Open-source*	Built-in hybrid search (vector + keyword). Module system for embedding models.	`BSD-3`
Chroma *Open-source*	Developer-friendly, LangChain integration. Great for prototyping. 2025 Rust rewrite.	`Apache 2.0`

Convergence: Vectors as a Data Type

$1.25B spent by Snowflake and Databricks acquiring PostgreSQL companies in 2024–25

The core argument: vectors are a data type, not a database category. Every major database now supports vector search as a built-in feature.

PostgreSQL + pgvector benchmarks at 471 QPS vs. Qdrant's 41 QPS at 99% recall on 50M vectors (pgvectorscale). Why run a separate Pinecone instance when your existing Postgres can do vector search?

But purpose-built still wins at scale: At billions of vectors, Qdrant, Pinecone, and Milvus outperform pgvector. The performance gap is real — it's just narrowing fast.

Binding constraint lens: If your binding constraint is operational simplicity (one database to manage), pgvector wins. If it's search latency at billion-scale, purpose-built wins.

04

RAG: The Killer App for Vector Databases

How retrieval-augmented generation turned vectors into critical infrastructure

Retrieval-Augmented Generation (RAG)

Why RAG Matters

LLMs are trained on general data and have a knowledge cutoff. RAG grounds their responses in your specific, current documents — reducing hallucination and enabling domain-specific answers without retraining. The vector database is the bridge between your organization's knowledge and the LLM.

Where Vector-Only RAG Breaks Down

Three failure modes that motivate Part 2 of today's lecture

Multi-Hop Reasoning

"Product A uses Component X. Component X requires Certification Y. Certification Y expires after 2 years." Vector search finds each chunk independently — it cannot traverse the logical chain to answer "How often must we recertify Product A?"

Entity Disambiguation

Your documents mention "Dr. Smith" 47 times — 23 references to an oncologist, 24 to a cardiologist. Vector similarity treats them identically. Structured relationships can distinguish them.

Global Synthesis

"What are the top compliance issues across all 1,000 audit reports?" Vector RAG retrieves the 5–10 most similar chunks. It cannot synthesize patterns across the entire corpus.

These are the problems graph databases solve. We'll explore them in Part 2.

Vector Search on AWS

Amazon Bedrock Knowledge Bases

Fully managed RAG pipeline — ingest documents, embed, store, and retrieve with a single API. Supports Aurora pgvector, OpenSearch, and Pinecone as backing stores. The easiest on-ramp.

Amazon Aurora PostgreSQL + pgvector

Add vector columns to your existing relational database. Use the same Postgres you already know. Best for teams that want one database for both transactional and semantic workloads.

Amazon OpenSearch Serverless

k-NN vector search built into OpenSearch. Serverless option auto-scales. Combines full-text + vector hybrid search. Production-proven at scale.

Amazon Neptune Analytics

Graph + vector in one service. Supports vector similarity search alongside graph traversals. We'll explore this in Part 2.

05

Policy Implications

Where vector databases meet public sector data challenges

Vector Databases in the Public Sector

FOIA & Document Search

Semantic search across millions of government documents. Find relevant records by meaning, not just keywords. Dramatically reduces FOIA response times.

Fraud Detection

Embed transaction patterns and claim descriptions. Identify anomalous submissions by semantic similarity to known fraud cases.

Legislative Analysis

Embed bills, committee reports, and regulatory text. Find similar provisions across jurisdictions. Track how language evolves across legislative sessions.

Constituent Services

RAG-powered chatbots that answer citizen questions using agency policy documents. Reduces call center volume while improving accuracy.

The Market in 2026

$2.5B

Vector DB
market size

Growing at ~28% CAGR

40yr

PostgreSQL
turns 40 in 2026

More relevant than ever

78%

Businesses feel
unprepared for GenAI

Data foundations are the gap

The key takeaway: vectors are becoming a standard feature of every database, just as JSON support did a decade ago. The question is not whether to use vector search, but where to run it.

Coming Up in Part 2

Graph Databases: Relationships Are First-Class Citizens

Graph theory from Euler to Neo4j

The labeled property graph model

Cypher and the new GQL ISO standard

Knowledge graphs and GraphRAG

The convergence: graph + vector working together